

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment: Comprehensive Algorithmic Analysis
Assignments 1 to 6

1 Introduction to Divide and Conquer

The **Divide and Conquer** paradigm is a fundamental algorithmic design technique based on multi-branched recursion. It breaks a complex problem into smaller, independent subproblems of the same type, solves them recursively, and subsequently combines their individual solutions to construct the solution to the original problem. The paradigm adheres to three canonical phases:

1. **Divide:** Partition the input problem into two or more disjoint subproblems of smaller size.
2. **Conquer:** Solve the subproblems recursively. If the subproblem size is sufficiently small (the base case), solve it directly without further subdivision.
3. **Combine:** Merge the solutions of the subproblems into a unified global solution.

In this laboratory report, we implement, benchmark, and analyze key algorithmic problems covering Divide and Conquer, data structures, high-resolution OS timing, selection order statistics, and loop complexity:

- **Assignment 1 (2D Closest Pair of Points):** Divide and Conquer $O(n \log n)$ geometric partitioning contrasted with an $O(n^2)$ exhaustive Brute Force comparison.
 - **Assignments 2 & 3 (Large Dataset & Microsecond Timing):** Generating 10^6 random positive integers, resolving duplicate collisions to nearest free values, and microsecond-level timing via monotonic Linux clocks.
 - **Assignment 4 (Order Statistics Selection):** Priority Queue (Heap) selection ($O(n \log k)$) versus Quicksort partition (Quickselect, expected $O(n)$) and full sorting ($O(n \log n)$).
 - **Assignment 5 (Cost of Execution & Loop Analysis):** Empirical operation counts and wall-clock execution for four nested/iterative loop constructs compared with asymptotic models ($O(n^2)$, $O(n)$, $O(n^3)$, $O(n \log n)$).
 - **Assignment 6 (Large Integer Multiplication):** Karatsuba divide-and-conquer multiplication ($O(n^{\log_2 3}) \approx O(n^{1.585})$) contrasted against conventional grade-school multiplication ($O(n^2)$).
-

2 Assignment 1: Closest Pair of Points in a 2D Plane

2.1 Problem Formulation and Theoretical Analysis

Given a set P of n points in a two-dimensional Euclidean plane, find two points $p_i, p_j \in P$ such that their Euclidean distance $d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ is minimized.

1. Brute-Force Approach: Computes the pairwise distance between every pair (p_i, p_j) for $1 \leq i < j \leq n$. There are $\binom{n}{2} = \frac{n(n-1)}{2}$ pairs, giving a time complexity of $\Theta(n^2)$.

2. Divide and Conquer Approach:

1. **Pre-sorting:** Points are pre-sorted by x -coordinate (P_x) and y -coordinate (P_y) in $O(n \log n)$ time.
2. **Divide:** The points are partitioned by a vertical line $x = x_{\text{mid}}$ into left half Q and right half R of size $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$.
3. **Conquer:** Recursively find the closest pair in Q with minimum distance δ_L and in R with minimum distance δ_R . Let $\delta = \min(\delta_L, \delta_R)$.
4. **Combine:** Construct a vertical strip containing points whose x -coordinates are within δ of x_{mid} . For each point in the strip (sorted by y), we only need to inspect at most 7 subsequent points.

The recurrence relation is $T(n) = 2T(n/2) + O(n)$, which resolves by the Master Theorem to $T(n) = \Theta(n \log n)$.

2.2 Implementation

```

1 import math, time, random
2
3 def dist(p1, p2):
4     return math.hypot(p1[0] - p2[0], p1[1] - p2[1])
5
6 # 1. Brute Force O(n^2)
7 def closest_brute_force(pts):
8     n = len(pts)
9     min_d, pair = float('inf'), None
10    for i in range(n):
11        for j in range(i + 1, n):
12            d = dist(pts[i], pts[j])
13            if d < min_d:
14                min_d, pair = d, (pts[i], pts[j])
15    return min_d, pair
16
17 # 2. Divide and Conquer O(n log n)
18 def closest_divide_conquer(pts):
19     px = sorted(pts, key=lambda p: p[0])
20     py = sorted(pts, key=lambda p: p[1])
21
22     def rec(px, py):
23         n = len(px)
24         if n <= 3:

```

```

25         return closest_brute_force(px)
26     mid = n // 2
27     mid_x = px[mid][0]
28     lx, rx = px[:mid], px[mid:]
29     set_lx = set(lx)
30     ly = [p for p in py if p in set_lx]
31     ry = [p for p in py if p not in set_lx]
32
33     d1, p1 = rec(lx, ly)
34     d2, p2 = rec(rx, ry)
35     d, pair = (d1, p1) if d1 < d2 else (d2, p2)
36
37     strip = [p for p in py if abs(p[0] - mid_x) < d]
38     for i in range(len(strip)):
39         for j in range(i + 1, min(i + 8, len(strip))):
40             sd = dist(strip[i], strip[j])
41             if sd < d:
42                 d, pair = sd, (strip[i], strip[j])
43     return d, pair
44
45 return rec(px, py)

```

2.3 Empirical Results and Performance Comparison

Execution times were measured using the high-resolution timer across increasing point set sizes:

Table 1: Empirical Timing Comparison: 2D Closest Pair of Points

Input Size (N)	Brute Force (μs)	Divide & Conquer (μs)	Solutions Match
50	619.91	403.25	True
100	2,165.96	668.08	True
200	7,036.75	1,250.83	True
400	31,929.28	3,124.95	True
800	132,358.18	7,665.49	True

Observation: For $N = 800$, the Divide and Conquer algorithm executes in 7.67 ms, whereas the Brute Force approach requires 132.36 ms (a speedup of over **17.2** $\times$). As N doubles, Brute Force runtime roughly quadruples ($O(n^2)$), while Divide and Conquer scales almost linearly ($O(n \log n)$).

3 Assignments 2 & 3: Dataset Creation, Duplicate Resolution & Microsecond Timing

3.1 Problem Statement and Design

- **Dataset Creation:** Populate a dynamic vector with at least 1,000,000 positive random integers (simulating POSIX 'lrnd48()' behavior).

- **Duplicate Detection:** Detect and print the total count of duplicate elements: $\text{Duplicates} = N - |\text{Unique}(A)|$.
- **Duplicate Resolution:** Reassign each duplicate value to its nearest free positive integral value (searching offsets $x \pm 1, x \pm 2, \dots$).
- **High-Resolution Timing (Assignment 3):** Output execution times formatted in microseconds (μs) and analyze Linux timing facilities.

3.2 High-Resolution Timing Discussion in Linux

Under Linux, three timing approaches provide varying degrees of precision:

1. **CLI Utility ('usr/bin/time -v'):** Provides macro-level elapsed wall-clock time, system/user CPU time, page faults, and peak resident memory.
2. **POSIX Monotonic Clocks ('clock_gettime(CLOCK_MONOTONIC'):** Directly samples hardware timers (TSC register on x86) with nanosecond resolution. Unlike 'CLOCK_REALTIME', it is strictly monotonic and unaffected by NTP server adjustments or system clock jumps.
3. **Python 'time.perf_counter()' :** Built upon the finest OS monotonic counter. Dividing elapsed nanos

3.3 Implementation

```

1 # High-Resolution Microsecond Timer
2 def timer(func, *args, **kwargs):
3     t0 = time.perf_counter_ns()
4     res = func(*args, **kwargs)
5     t1 = time.perf_counter_ns()
6     return res, (t1 - t0) / 1000.0
7
8 # 1. Populate vector with 1,000,000 positive integers
9 def generate_dataset(n=1_000_000, max_val=1_500_000):
10     return [random.randint(1, max_val) for _ in range(n)]
11
12 # 2. Duplicate detection and round-off to nearest free integer
13 def resolve_duplicates(arr):
14     seen = set()
15     duplicates_count = 0
16     resolved = []
17     occupied = set(arr)
18
19     for x in arr:
20         if x not in seen:
21             seen.add(x)
22             resolved.append(x)
23         else:
24             duplicates_count += 1
25             step = 1
26             while True:
27                 c_up = x + step
28                 if c_up not in occupied:

```

```
29         occupied.add(c_up)
30         resolved.append(c_up)
31         break
32     c_down = x - step
33     if c_down > 0 and c_down not in occupied:
34         occupied.add(c_down)
35         resolved.append(c_down)
36         break
37     step += 1
38     return duplicates_count, resolved
```

3.4 Empirical Output

```
Dataset Size: 1,000,000 positive integers
Generation Time: 588,580.11 us (588.58 ms)
Duplicates Found in 1M dataset: 270,475
Duplicates in 50k sample: 806
Resolution Time (50k sample): 20,190.63 us (20.19 ms)
Remaining duplicates after resolution: 0
```

4 Assignment 4: Finding the Element at Index i in Sorted Order

4.1 Algorithm Descriptions and Theoretical Comparison

Given an unsorted array A of size n and an index i ($0 \leq i < n$), find the element that would appear at index i if A were fully sorted:

1. Priority Queue (Heap Selection):

- For $i < n/2$: Maintain a max-heap of size $i + 1$ containing the smallest $i + 1$ elements. Time complexity: $O(n \log(i + 1))$.
- For $i \geq n/2$: Maintain a min-heap of size $n - i$ containing the largest $n - i$ elements. Time complexity: $O(n \log(n - i))$.
- *Trade-off*: When $i \approx n/2$ (median), complexity reaches $O(n \log n)$.

2. Quicksort Partition (Quickselect):

Partitions the array around a pivot element. If the pivot index $p = i$, the target is found. Otherwise, the algorithm recurses into only one partition half. Expected time complexity: $O(n)$, worst-case $O(n^2)$.

3. Full Sorting Baseline:

Sort the entire array via Timsort in $O(n \log n)$ time and access $A[i]$ in $O(1)$.

4.2 Implementation

```

1 import heapq
2
3 # 1. Priority Queue Selection
4 def select_heap(arr, i):
5     n = len(arr)
6     if i < n // 2:
7         return heapq.nsmallest(i + 1, arr)[-1]
8     else:
9         return heapq.nlargest(n - i, arr)[-1]
10
11 # 2. Quickselect using Quicksort Partition Routine
12 def quickselect(arr, i):
13     def partition(a, l, r):
14         pivot = a[r]
15         p = l
16         for j in range(l, r):
17             if a[j] <= pivot:
18                 a[p], a[j] = a[j], a[p]
19                 p += 1
20         a[p], a[r] = a[r], a[p]
21         return p
22
23     def select(a, l, r, k):
24         if l == r:
25             return a[l]
26         p = partition(a, l, r)
27         if k == p:
28             return a[p]
29         elif k < p:
30             return select(a, l, p - 1, k)
31         else:
32             return select(a, p + 1, r, k)
33
34     return select(arr.copy(), 0, len(arr) - 1, i)
35
36 # 3. Full Sort Baseline
37 def select_sort(arr, i):
38     return sorted(arr)[i]

```

4.3 Empirical Comparison Across Index Locations

Testing on an unsorted array of size $N = 50,000$:

Table 2: Selection Time Comparison for $N = 50,000$ Across Different Target Indices i

Target Index i	Heap (μ s)	Quickselect (μ s)	Sort (μ s)	Fastest Method	All Match
0 (Min)	927.85	27,941.30	13,153.62	Heap	True
10	1,670.34	28,237.52	14,423.20	Heap	True
25,000 (Median)	132,636.20	29,368.93	12,444.46	Sort	True
49,989	1,512.62	12,418.63	13,670.59	Heap	True
49,999 (Max)	763.77	10,999.95	11,549.42	Heap	True

Analysis: When i is close to the extremes ($i = 0, 10$ or $i \approx n - 1$), the Priority Queue method is exceptionally fast (< 1.7 ms) because the heap size k is small. However, for the median ($i = 25,000$), maintaining a heap of size 25,000 incurs substantial overhead (132.6 ms), making Quickselect and full sorting significantly superior.

5 Assignment 5: Cost of Execution and Asymptotic Loop Analysis

5.1 Analytical Complexity Derivations

We analyze the exact number of times statement $x = x + 1$ is executed:

(a) **Code 1:**

$$T_1(n) = \sum_{i=1}^n \sum_{j=1}^i 1 = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

(b) **Code 2:** The question sheet writes ' $j = n/2$;' inside ' $\text{while } (j \neq 1)$ '. If executed literally with $n \geq 2$, j remains unchanged at $n/2$ indefinitely, resulting in an infinite loop. The intended algorithmic formulation is logarithmic halving ' $j = j // 2$ ':

$$T_2(n) = \sum_{k=0}^{\lfloor \log_2 n \rfloor} \left\lfloor \frac{n}{2^k} \right\rfloor \leq n \sum_{k=0}^{\infty} \frac{1}{2^k} = 2n = \Theta(n)$$

(c) **Code 3:**

$$T_3(n) = \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^i 1 = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

(d) **Code 4:** The outer loop divides i by 2 until $i < 1$ ($\lfloor \log_2 n \rfloor + 1$ iterations). The inner loop runs n times per outer step:

$$T_4(n) = (\lfloor \log_2 n \rfloor + 1) \times n = \Theta(n \log n)$$

5.2 Implementation

```

1 def code1(n):
2     x = 0
3     for i in range(1, n + 1):
4         for j in range(1, i + 1):
5             x += 1
6     return x
7
8 def code2(n):
9     x, j = 0, n
10    while j >= 1:
11        for i in range(1, j + 1):

```

```

12         x += 1
13         j = j // 2 # Intended logarithmic halving
14     return x
15
16 def code3(n):
17     x = 0
18     for i in range(1, n + 1):
19         for j in range(1, i + 1):
20             for k in range(1, i + 1):
21                 x += 1
22     return x
23
24 def code4(n):
25     x, i = 0, n
26     while i >= 1:
27         for j in range(1, n + 1):
28             x += 1
29         i = i // 2
30     return x

```

5.3 Empirical Step Counts and Timing Benchmarks

Table 3: Exact Operation Counts vs. Theoretical Closed-Form Equations

Input (n)	Code 1: $n(n+1)/2$	Code 2: $\sum \lfloor n/2^k \rfloor$	Code 3: $n(n+1)(2n+1)/6$	Code 4: $(\lfloor \log_2 n \rfloor + 1)n$
10	55	18	385	30
50	1,275	97	42,925	150
100	5,050	197	338,350	300

Table 4: Empirical Execution Times (μ s) for the Four Loop Snippets

Input (n)	Code 1 $O(n^2)$	Code 2 $O(n)$	Code 3 $O(n^3)$	Code 4 $O(n \log n)$
50	84.84	6.43	3,283.11	22.49
100	472.45	15.35	22,867.57	42.39
200	1,155.02	18.72	148,196.00	75.33
300	2,065.12	27.72	505,136.97	158.19

6 Assignment 6: Large Integer Multiplication (Karatsuba Algorithm)

6.1 Theoretical Derivation and Divide-and-Conquer Recurrence

Given two large n -digit integers X and Y , write them as:

$$X = x_1 \cdot 10^m + x_0, \quad Y = y_1 \cdot 10^m + y_0 \quad (m = \lfloor n/2 \rfloor)$$

Their full product is given by:

$$X \cdot Y = x_1 y_1 \cdot 10^{2m} + (x_1 y_0 + x_0 y_1) \cdot 10^m + x_0 y_0$$

1. Conventional Multiplication: Naively computing the 4 products $(x_1 y_1, x_1 y_0, x_0 y_1, x_0 y_0)$ results in the recurrence $T(n) = 4T(n/2) + O(n) = O(n^2)$.

2. Karatsuba Algorithm: Karatsuba cleverly computes only **3 multiplications**:

$$\begin{aligned} z_0 &= x_0 \cdot y_0 \\ z_2 &= x_1 \cdot y_1 \\ z_1 &= (x_0 + x_1)(y_0 + y_1) - z_0 - z_2 = x_1 y_0 + x_0 y_1 \end{aligned}$$

Then, $X \cdot Y = z_2 \cdot 10^{2m} + z_1 \cdot 10^m + z_0$. The recurrence relation becomes:

$$T(n) = 3T(n/2) + O(n)$$

By the Master Theorem ($a = 3, b = 2, d = 1$), since $\log_2 3 \approx 1.585 > 1$, the overall time complexity is:

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right)$$

6.2 Implementation

```

1 # 1. Conventional Large Integer Multiplication O(n^2)
2 def mult_conventional(x, y):
3     s_y = str(y)
4     total = 0
5     for i, digit in enumerate(reversed(s_y)):
6         total += (x * int(digit)) * (10 ** i)
7     return total
8
9 # 2. Karatsuba Divide and Conquer Multiplication O(n^1.585)
10 def karatsuba(x, y):
11     if x < 10 or y < 10:
12         return x * y
13     n = max(len(str(x)), len(str(y)))
14     if n <= 16:
15         return x * y
16     m = n // 2
17     p = 10 ** m
18     x1, x0 = divmod(x, p)
19     y1, y0 = divmod(y, p)
20     z0 = karatsuba(x0, y0)
21     z2 = karatsuba(x1, y1)
22     z1 = karatsuba(x0 + x1, y0 + y1) - z0 - z2
23     return z2 * (10 ** (2 * m)) + z1 * p + z0

```

6.3 Empirical Scaling Benchmark

Table 5: Empirical Multiplication Performance Across Increasing Digit Lengths

Digit Length (d)	Conventional (μ s)	Karatsuba (μ s)	Empirical Speedup	Matches $X \times Y$
32	46.43	21.12	2.20×	True
64	75.30	81.21	0.93×	True
128	167.84	113.15	1.48×	True
256	611.74	302.27	2.02×	True
512	2,279.90	970.40	2.35×	True
1,024	15,247.29	2,985.25	5.11×	True

Discussion: At $d = 64$ digits, recursion overhead is comparable to grade-school multiplication. However, as d scales to 1,024 digits, conventional multiplication time increases dramatically due to $O(n^2)$ growth (15.25 ms), whereas Karatsuba scales at $O(n^{1.585})$ (2.99 ms), achieving a **5.11×** speedup.

7 Conclusion

Through theoretical analysis and empirical benchmarking of assignments 1 through 6, we demonstrated:

1. **Divide and Conquer Efficiency:** For both 2D Closest Pair ($O(n \log n)$ vs $O(n^2)$) and Karatsuba integer multiplication ($O(n^{1.585})$ vs $O(n^2)$), multi-branch recursion significantly outperforms naive algorithms as problem size grows.
2. **Trade-offs in Selection:** Priority queue selection is optimal for finding boundary elements ($i \approx 0$ or $i \approx n - 1$), while Quickselect and sorting prevail for median order statistics.
3. **Precision Timing:** Python monotonic timers provide sub-microsecond resolution, verifying asymptotic complexity models empirically.